Solution: Merge Two Binary Trees (medium)

Problem Statement

Given two binary trees, `root1` and `root2`, merge them into a single, new binary tree.

If two nodes from the given trees share the same position, their values should be summed up in the resulting tree. If a node exists in one tree but not in the other, the resulting tree should have a node at the same position with the value from the existing node.

Examples

Example 1:

Trees:

```
Tree 1:      1              Tree 2:       1
           /   \                        /   \
          3     2                      2     3

Merged:        2
             /   \
            5     5
```

Justification: The root nodes of both trees have the value `1`, so the merged tree's root has a value of `1 + 1 = 2`. For the left child, `3 + 2 = 5` and for the right child, `2 + 3 = 5`.

Example 2:

Trees:

```
Tree 1:      5              Tree 2:       3
           /   \                        /   \
          4     7                      2     6

Merged:        8
             /   \
            6    13
```

Justification: The root nodes have values `5` and `3` respectively. So, the merged tree's root value becomes `5 + 3 = 8`. The left child is `4 + 2 = 6` and the right child is `7 + 6 = 13`.

Example 3:

Trees:

```
Tree 1:      2              Tree 2:      2
              \                         /
               5                       3

Merged:        4
             /   \
            3     5
```

Justification: The root nodes have values `2` each, so the merged tree's root value is `2 + 2 = 4`. Tree 1 doesn't have a left child, so we take the left child of Tree 2 as it is, which is `3`. Similarly, Tree 2 doesn't have a right child, so the merged tree's right child is the same as Tree 1, which is `5`.

Example 4:

Trees:

```
Tree 1:      10             Tree 2:       10
           /    \                       /    \
          5      15                    6      16

Merged:       20
             /   \
            11    31
```

Justification: The root nodes have the value `10`, so they add up to `20` for the merged tree. The left child values add up to `5 + 6 = 11` and the right child values sum up to `15 + 16 = 31`.

Solution

The most straightforward approach to solve this problem is to use a recursive algorithm. Start by checking if either of the nodes from the two trees at the current position is `null`. If so, return the non-null node as it is; if both are null, return null. If both nodes exist, create a new node with the sum of their values.

The next steps are recursive calls. Call the function recursively for the left and right children of the two current nodes and set the left and right children of the new node based on these calls.

1. **Base Case Checks:**
   - If `root1` is null, return `root2`.
   - If `root2` is null, return `root1`.
2. **Node Value Summation:** If both `root1` and `root2` are not null, create a new node with a value equal to the sum of `root1` and `root2`'s values.
3. **Recursive Calls for Child Nodes:**
   - Recursively call `mergeTrees` for the left children of `root1` and `root2`, and set the left child of the new node to the result of this recursive call.
   - Similarly, recursively call `mergeTrees` for the right children of `root1` and `root2`, and set the right child of the new node to the result.
4. **Return the Merged Node:** The new node now represents the merged result of `root1` and `root2` at this position in the tree. Return this node.
5. **Termination:** The recursion will terminate once all nodes in both trees have been traversed. The return value of the initial call to `mergeTrees` will be the root of the newly merged binary tree.

This approach works well because we are addressing every scenario: 1) both nodes are null, 2) one node is null and the other is not, and 3) both nodes exist. By handling these cases, we ensure that the new tree is correctly constructed.

Algorithm Walkthrough

For example, let's take the following example trees: [1, 2, 3, 4, 5] and [1, 2, 3, null, 5]:

```
Tree 1:      1              Tree 2:       1
           /   \                        /   \
          2     3                      2     3
         / \                            \
        4   5                            5

Merged:
               2
             /   \
            4     6
           / \
          4   10
```

**Here is the visual walkthrough of the algorithm:*

1. **Start at the Root of Both Trees (1 and 1):**
   - Since both roots are not null, create a new node with the sum of both values: `1 + 1 = 2`.
   - Recursively merge the left children and right children.
2. **Merge Left Children (2 and 2):**
   - Both nodes are not null, so create a new node with the sum: `2 + 2 = 4`.
3. **Merge Left's Left Children (4 and null):**
   - Since the second node is null, return the first node (4).
4. **Merge Left's Right Children (5 and 5):**
   - Both nodes are not null, so create a new node with the sum: `5 + 5 = 10`.
5. **Merge Right Children (3 and 3):**
   - Both nodes are not null, so create a new node with the sum: `3 + 3 = 6`.
   - Merge the left children (null and null) and the right children (null and null).
   - Both left and right children are null, so they remain null.
6. **Construct the Merged Tree:**
   - Now, we construct the merged tree using the new nodes created during the merging process.

Here is the visual representation of the algorithm:

We will have to merge these two trees to make a new one

Solution:

We will start from the root of both trees and pass the nodes to the MergeTrees function

A new node will be created from the values of highlighted nodes by adding them

Now we will make the left side of **new node 2** by again calling MergeTrees function and passing the highlighted nodes

A new node will be created from the values of highlighted nodes by adding them

Now we will make the left side of **new node 4** by again calling MergeTrees function and passing the highlighted nodes

Since there is no left node for Tree2 so NULL will be passed to the MergeTrees and at the end, we will return Tree1 node i.e. node 4

Since exit condition is called so we will traverse back to **node 2 of both trees**

And now the **new node 4** will have a left child which is 4

Now, we will move to the right of both trees to make the right side of **new node 4**

A new node will be created from the values of highlighted nodes by adding them

Now, we will move to the left of both trees to make the left side of **new node 10**

Since both them are NULL so NULL will be returned and hence the left side of **new node 10 will be NULL**

Now, we will traverse back to node 5 of both trees and move to the right of them and NULL will be returned as both have no right child.
So the **new node 10** will be:

At the end, we will return the **new node 10**

Since we are traversing back so we will be now at node 2 of both trees and now the **new node 4** will have a right child whcih is 10

Since the **new node 4** have its left and right child set so we will return **new node 4** and we will traverse back and reach node 1 of both trees

The new nodes will be linked while traversing back and will look like:

Now, we will move to right of Tree1 and Tree2

A new node will be created from the values of highlighted nodes by adding them

Now, we will move to the left of both trees to make the left side of **new node 6**

Since both them are NULL so NULL will be returned and hence the left side of **new node 6 will be NULL**

Now, we will traverse back to node 3 of both trees and move to the right of them and NULL will be returned as both have no right child.
So the **new node 6** will be:

At the end, we will return the **new node 6**

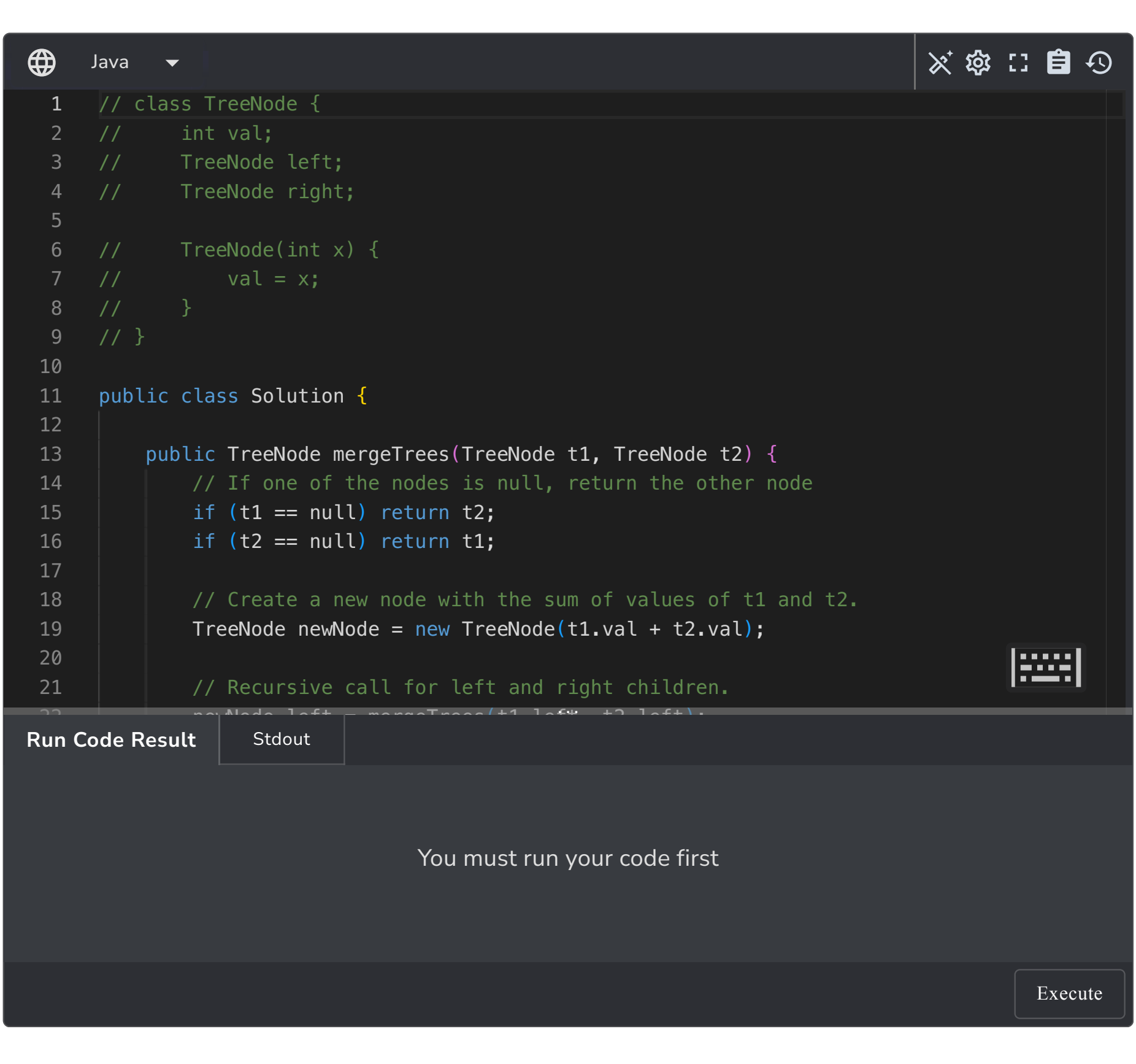
We will traverse back to node 1 of both trees and hence the all new nodes will be linked

New node 6 will be assigned to the right of **new node 2**

Merge Two BT

Code

Here is the code for this algorithm:



```java
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;

//     TreeNode(int x) {
//         val = x;
//     }
// }

public class Solution {

    public TreeNode mergeTrees(TreeNode t1, TreeNode t2) {
        // If one of the nodes is null, return the other node
        if (t1 == null) return t2;
        if (t2 == null) return t1;

        // Create a new node with the sum of values of t1 and t2.
        TreeNode newNode = new TreeNode(t1.val + t2.val);

        // Recursive call for left and right children.
```

You must run your code first

Complexity Analysis

- Time Complexity: $O(min(n, m))$, where n and m are the numbers of nodes in the first and second trees respectively.
- Space Complexity: $O(min(n, m))$, as we're creating a new tree with that many nodes.

← Previous
Merge Two Binary Trees (Medium)

Next →
Introduction To Hashing

☐ Mark as Completed